Thai Province Guide

A grounded question-answering app and tool-using agent over Thailand's 77 provinces

Thanawat (67070236) · KMITL · LLM-application capstone · November 2026

Repository: github.com/PELY334/thai-province-guide · model: gemma-4-E4B-it via the class proxy

1. Problem and who it is for

People who want a quick fact about a Thai province (a student writing a report, a teacher preparing a quiz, a traveller planning a trip) usually ask a chatbot. A small model answers confidently and is usually wrong: asked about provincial symbols, rivers and parks, the same model we use (gemma-4-E4B-it) answered only **2 of 34** test questions correctly without retrieval, said "I don't know" on 16, and confidently invented the other 16 (Lampang's seal "features a horse"; Phuket's area is "484 km^2"). The useful version of this product is one that answers *only* from a known source, shows the exact passage, and says "I can't answer that" when the source is silent.

Thai Province Guide does that over the English Wikipedia article for each of the 77 provinces. It also handles questions that need two facts and arithmetic ("how much larger is Chiang Mai than Kanchanaburi?") with an agent that searches twice and uses a calculator, rather than letting the model do sums in its head. The corpus is new to this course: the reference app answers questions about the course's own stack and has never seen these articles.

2. Design

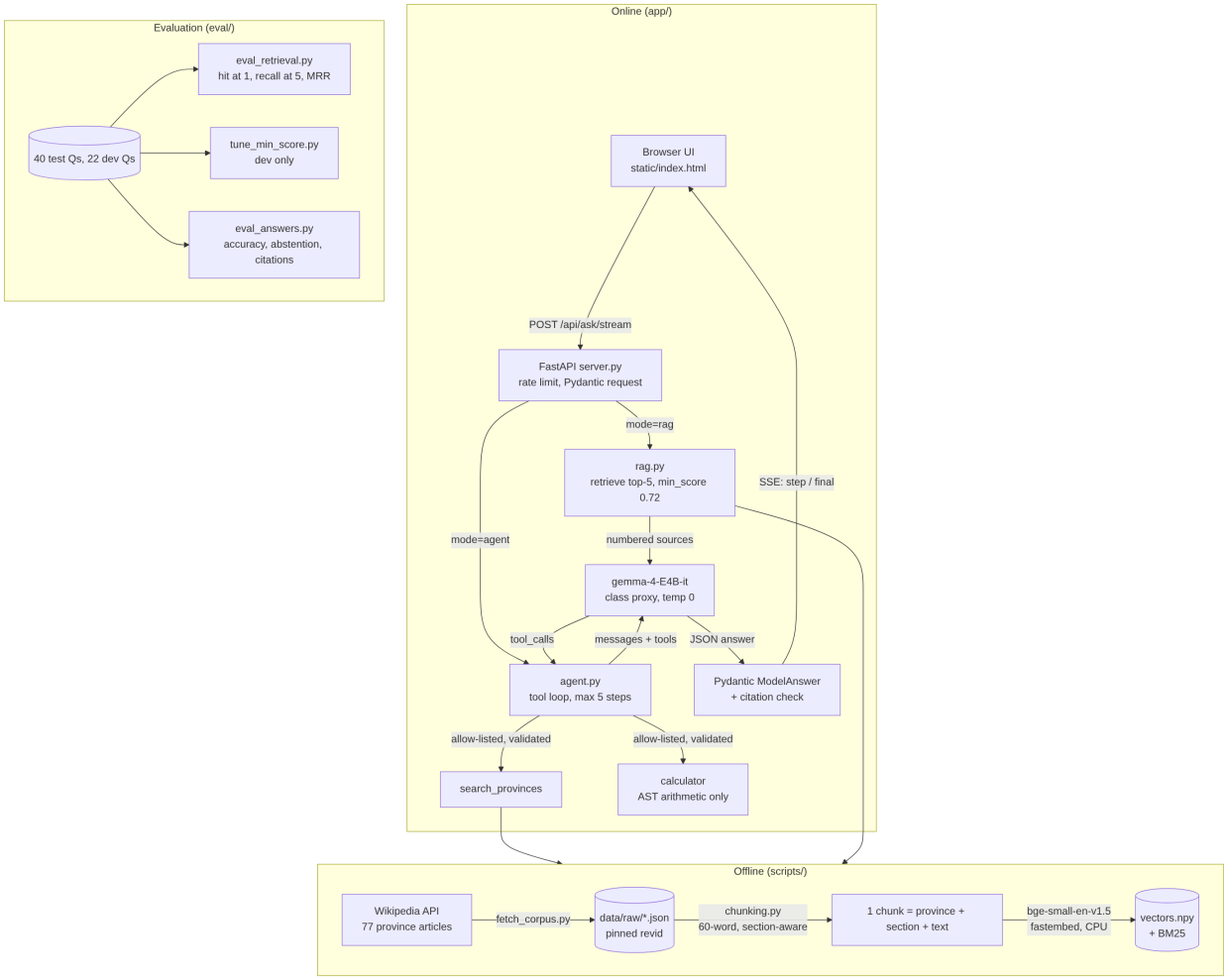


Figure 1. Architecture. Offline: fetch, chunk, embed. Online: FastAPI routes a question to single-shot RAG or to the agent; every model output passes through Pydantic and a citation check before it is streamed back. Evaluation scripts read the same app code.

Techniques and why

Course technique	How it is used here	Why
Structured output + validation	The model must return {answer, citations, abstain}. Pydantic rejects an answer without a citation; one repair turn shows the model its own error; citations to sources we did not send are dropped, and an answer left uncited becomes an abstention.	A citation is only worth something if it is checked by code, not trusted.
RAG with citations	Section-aware chunks of at most 60 words (never cutting a sentence), embedded with bge-small-en-v1.5 (local, CPU) as "<Province> province. <Section>. <text>". Top 5 by cosine; numbered sources in the prompt.	Local embeddings keep retrieval deterministic and free; the province prefix fixes passages that only say "the province".
min_score floor	Hits under cosine 0.72 are dropped; if none survive, the app abstains without calling the LLM.	Cheap first line of defence for off-topic questions; tuned on a dev set.
Agent + function calling	Native OpenAI tool calls; tools search_provinces and calculator. Allow-list, Pydantic arguments, tool errors returned as text, tool output wrapped as untrusted data. Stops on a final answer, after 5 model turns, or on a repeated identical call.	Multi-hop arithmetic questions need two lookups and exact maths.
Sandboxed execution	calculator walks a whitelisted Python AST (numbers, + - * / % and small powers); no eval(), no names, no calls.	The model never runs code; it only gets arithmetic.
Served interface	FastAPI: /api/ask (JSON) and /api/ask/stream (SSE: one event per agent step, then the answer). Per-IP rate limit, upstream errors mapped to 429/502/504, stops on client disconnect.	Users see the agent's steps as they happen.
Evaluation	40-question labelled test set, 22-question dev set, retrieval metrics, end-to-end answer metrics, five systems compared.	Section 3.

All configuration comes from the environment (.env, git-ignored); the OpenAI client reads OPENAI_API_KEY and OPENAI_BASE_URL itself; temperature is 0 for every call. 22 offline unit tests cover chunking, BM25, the floor, JSON repair, the calculator's refusals, the tool allow-list, the agent's stop conditions (with a scripted fake model) and the API.

3. Evaluation

Test set (data/eval/questions.jsonl): 40 questions written by hand from the corpus. 23 single-fact questions; 7 "reverse" questions that do not name the province ("Which province's slogan calls it the coldest place in Siam?"), which make retrieval harder; 4 multi-hop questions that need two articles and arithmetic; and 6 questions the corpus cannot answer (a restaurant recommendation, an exchange rate, the current governor of Krabi). Each answerable item has gold answer aliases and a gold *evidence string* that must appear in a retrieved chunk, so relevance does not depend on chunk size. A script checks every evidence string exists in the corpus.

Dev set (22 questions, 12 answerable and 10 unanswerable, disjoint from test) is used only to choose min_score. **Metrics**: retrieval hit@1, recall@5 and MRR; answer accuracy on answerable questions (a fixed string rule: a gold alias appears in the answer, commas in numbers ignored; no LLM judge, so it is reproducible); correct-abstention rate on unanswerable questions; citation precision (share of cited passages that contain gold evidence). **Baselines**: the same model with no retrieval (closed-book), BM25-only retrieval, and the naive RAG default of 250-word chunks with no floor.

3.1 Retrieval (test set, 34 answerable questions, k = 5)

Retriever	Chunk words	Chunks	hit@1	recall@5	MRR
BM25	250	1503	0.618	0.868	0.731
dense (bge-small)	250	1503	0.882	0.897	0.897
hybrid RRF	250	1503	0.765	0.912	0.828
BM25	120	1947	0.647	0.868	0.751
dense (bge-small)	120	1947	0.853	0.897	0.882
hybrid RRF	120	1947	0.735	0.941	0.834
BM25	60	3047	0.647	0.853	0.723
dense (bge-small)	**60**	**3047**	**0.765**	**0.971**	**0.846**
hybrid RRF	60	3047	0.765	0.897	0.819

Measured improvement over the naive default. Moving from 250- to 60-word chunks raised dense recall@5 from 0.897 to **0.971**: the gold passage is in the top five for 33 of 34 questions instead of 30–31. hit@1 falls (0.882 to 0.765), but the model reads all five passages, so recall@5 is the number that matters. Dense beat BM25 at every size; hybrid RRF did not beat dense here, because the weaker BM25 ranking pulls good dense hits down, so we ship the simpler retriever.

3.2 Choosing min_score

A retrieval-only sweep (Figure 2) shows the problem: top-1 cosines of answerable questions (0.69–0.88) and unanswerable ones (0.64–0.73) overlap, so any floor trades recall for early abstention. The retrieval-only optimum on dev (0.73) would have made 15% of answerable test questions unretrievable. We therefore tuned the floor end-to-end: run the real RAG pipeline on the dev set for each candidate and score what the user sees; ties go to the higher floor (same quality, fewer LLM calls).

Floor	Dev overall (22 Qs)	Dev Qs that never reach the LLM
0.00	95.5%	0.0%
0.60	95.5%	9.1%
0.65	100.0%	13.6%
0.68	100.0%	22.7%
0.70	100.0%	31.8%
0.72 (chosen)	**100.0%**	**40.9%**
0.73	95.5%	50.0%
0.75	90.9%	54.5%

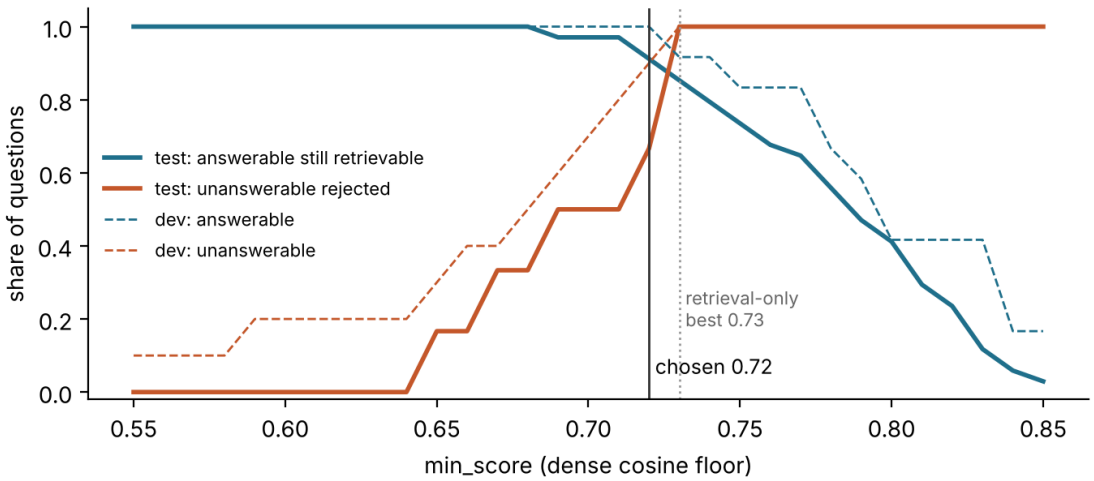


Figure 2. Retrieval-level effect of the floor. Solid = test, dashed = dev. A question is "still retrievable" if a gold passage remains in its filtered top 5.

3.3 End-to-end answers (test set: 34 answerable + 6 unanswerable)

System	Answerable correct	Single	Rev.	Multi + calc	Unans. abstained	Citation precision	Tool calls
Closed-book (baseline)	2/34 (5.9%)	2/23	0/7	0/4	6/6	–	0.0
RAG, naive (250 w, no floor)	31/34 (91.2%)	23/23	5/7	3/4	6/6	75.0%	0.0
RAG, 60 w, no floor	31/34 (91.2%)	23/23	7/7	1/4	6/6	80.5%	0.0
RAG shipped (60 w, 0.72)	**30/34 (88.2%)**	**23/23**	**5/7**	**2/4**	**6/6**	**84.2%**	**0.0**
Agent shipped (tools)	**30/34 (88.2%)**	**21/23**	**5/7**	**4/4**	**6/6**	**82.5%**	**1.1**

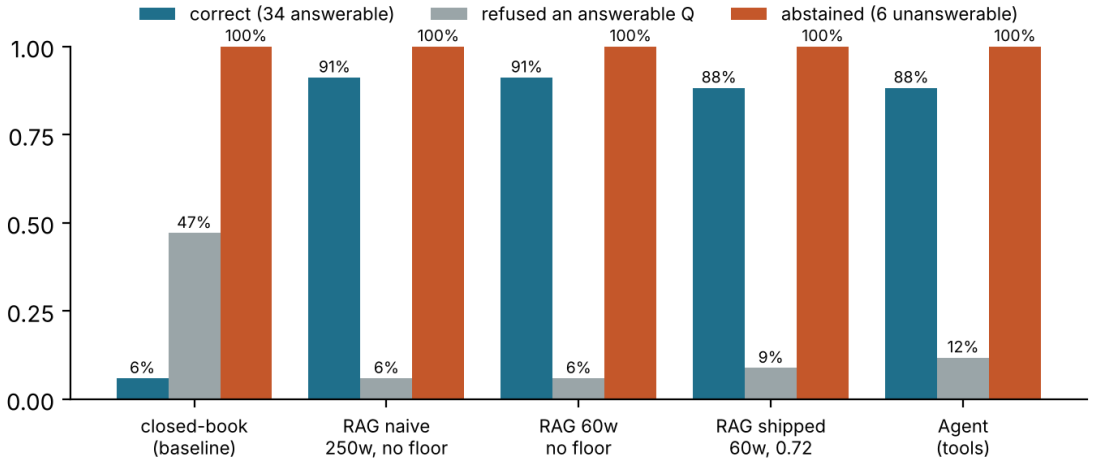


Figure 3. Correct answers and refusals on the 34 answerable questions, and abstention on the 6 unanswerable ones. The closed-book model "abstains" by refusing almost everything.

4. Results and honest analysis

- **Retrieval is the whole game.** The same model goes from 5.9% to 88–91% accuracy when it is given the passages. Every system abstained on all six unanswerable questions, but the closed-book model does it by saying "I don't know" to almost everything: it also refused 16 answerable questions and confidently got 16 more wrong. With retrieval, confidently wrong answers drop to 0–1 of 34; the remaining misses are abstentions.
- **The agent earns its cost on multi-hop questions only.** It solved 4/4 arithmetic questions (two searches + calculator) where single-shot RAG got 1–3/4, and RAG's misses include doing the subtraction in its head and getting it wrong ("20,736 - 10,589 = 9,147"). On single facts the agent is slightly worse (21/23 vs 23/23) because three times it answered without searching at all (below). Overall the two shipped systems tie at 30/34; the UI therefore offers both, with the agent as default for its multi-hop strength.
- **The floor is a cost and safety guard, not an accuracy gain.** On test, 0.72 stopped 4 of the 6 unanswerable questions before any LLM call (6 of 40 questions overall) and kept abstention at 6/6, but cost two answerable questions and fixed one by accident: 31 to 30. Gemma abstains well on its own when the sources are irrelevant, so most of the protection comes from the prompt + validation.
- **Chunk size helps retrieval more than answers.** 60-word chunks win clearly on recall@5 and on citation precision (75.0% to 80.5%), but answer accuracy ties with 250-word chunks (31/34 each): bigger chunks lose some reverse questions and win some multi-hop ones.
- **Small numbers.** One question is 2.9 points. Differences of one or two questions (RAG vs agent overall; 60 vs 250 words at the answer level) are within noise. Re-running all systems gave identical correct/incorrect outcomes on all 40 questions (temperature 0); only three answers changed wording.

Failure cases

- **q29, a fact split from its subject.** "Which dam in Tak created the largest artificial lake?" The 60-word chunk containing "the artificial lake… is the largest in Thailand" does not contain "Bhumibol Dam", which

ends the previous chunk; with the floor that chunk is cut, so the model correctly refuses to name the dam. This is the price of small chunks; overlapping windows are the obvious fix.

- **q16, q20, q22, the agent did not look.** On three easy questions the agent replied "can't answer" with zero tool calls; RAG answered all three. Forcing the first turn to call search_provinces (tool_choice="required") should fix this; we have not changed the shipped system after seeing test results, so this is listed as next work to be validated on dev.
- **q33, the source disagrees with itself.** The Nakhon Ratchasima article says 20,494 km^2 in one section and 20,736 km^2 in another. The eval surfaced this; the gold label now accepts both differences.
- **A bug in our own metric.** The first "I don't know" detector for the closed-book baseline missed "I do not have specific information", under-counting its abstentions (4/6 instead of 6/6). Found by reading outputs, fixed, unit-tested and re-scored without new model calls.
- **q13, our own label was wrong.** Bangkok's seal also shows Indra on the elephant Airavata. The agent said "Bangkok" and was marked wrong. We fixed the label and recorded both fixes in the question's note field; eval_answers.py --rescore re-scores saved answers against fixed labels without new model calls.
- **q11, the floor's false abstention.** "Which province's slogan calls it the coldest place in Siam?" has a best cosine of 0.689, under the floor, so the shipped RAG refuses; without the floor it answers Loei correctly.
- **Found in manual testing: a cited but wrong quantity.** "How much larger is Chiang Mai than Lamphun?" The Lamphun article gives no total area, only "total forest area is 2,588 km^2"; the agent subtracted that and answered 19,723 km^2 with a citation. The citation makes the mistake visible to a careful reader, but the system did not catch it. Not in the test set; it shows the set is easier than real use. Next: verify that the cited sentence states the quantity that was asked for.

5. Limitations and next steps

- 40 test questions written by one person; no second annotator, so label errors like q13 can remain. Next: 100+ questions and a second labeller, with agreement reported.
- Wikipedia extracts drop infoboxes and tables, so populations and many statistics are missing; the app abstains on them, which is correct but disappointing. Next: load infobox data into SQLite and give the agent a read-only, allow-listed SQL tool.
- Cosine floors are coarse. Next: a cross-encoder reranker score as the abstention signal, and a separate floor for the agent's short keyword queries.
- The proxy's bge-m3 embedding backend is implemented but building a 2,000-chunk index through the shared proxy took over 30 s per batch during the project, so the comparison with bge-small was not run.
- English only. Most users of a Thai-province guide would also ask in Thai; bge-m3 is multilingual and would be the first step.

Reproducing every number: `scripts/build_index.py --chunk-words 60 120 250`, `eval/eval_retrieval.py`, `eval/tune_min_score.py`, `eval/eval_answers.py`, then `eval/make_tables.py`. Per-question outputs for every system are committed in eval/results/.